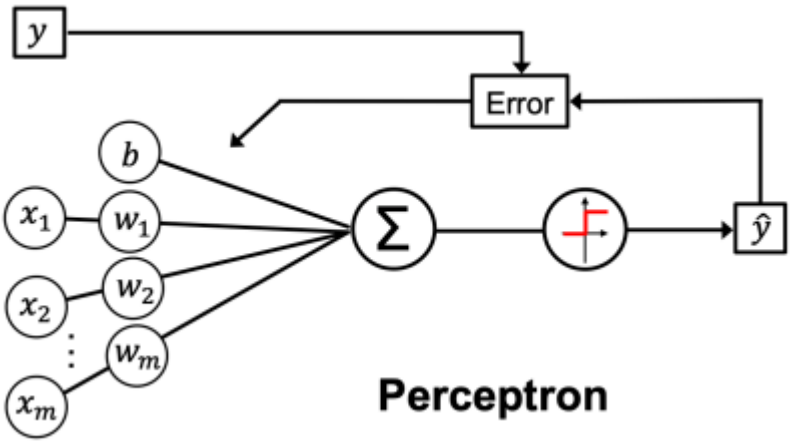
M2 Math Note

Chung-En (Johnny) Yu
Department of Intelligent Systems and Robotics, University of West Florida

> **Tip**
>
> This math note serves as supplementary content for the lecture. Students are encouraged to read it in conjunction with the lecture.

Perceptron

The Perceptron is a binary classifier that classifies data points into two categories by finding a decision boundary. The Perceptron algorithm works well with linearly separable data and uses a step activation function to predict outputs.



Artificial neurons

The net input z is computed as:

$$z = \sum_j w_j x_j + b = \mathbf{w}^T \mathbf{x} + b$$

Where:

- $\mathbf{x} = [x_1 \dots x_m]$ represents the input features.
- $\mathbf{w} = [w_1 \dots w_m]$ are the weights.
- b is the bias term.

What is net input?

- It is a linear combination of the weights that are connecting the input layer to the output layer.

- It can be written as a step activation function: $\sigma(z) = z$ for Perceptron.
- It is also the predicted output $\hat{y} = \sigma(z)$ for Perceptron.

Update rule

The Perceptron learning rule updates weights and bias iteratively based on misclassified samples:

$$w_j := w_j + \eta \cdot (y - \hat{y}) \cdot x_j$$
$$b := b + \eta \cdot (y - \hat{y})$$

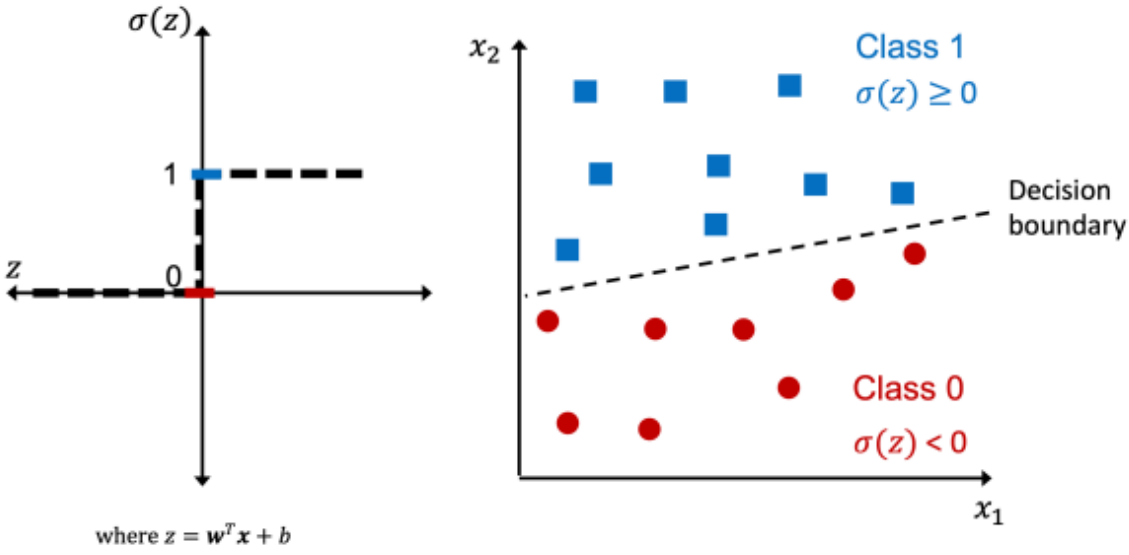
Where:

- η is the learning rate.
- y is the true label.
- $\hat{y}$ is the predicted output.

Threshold function

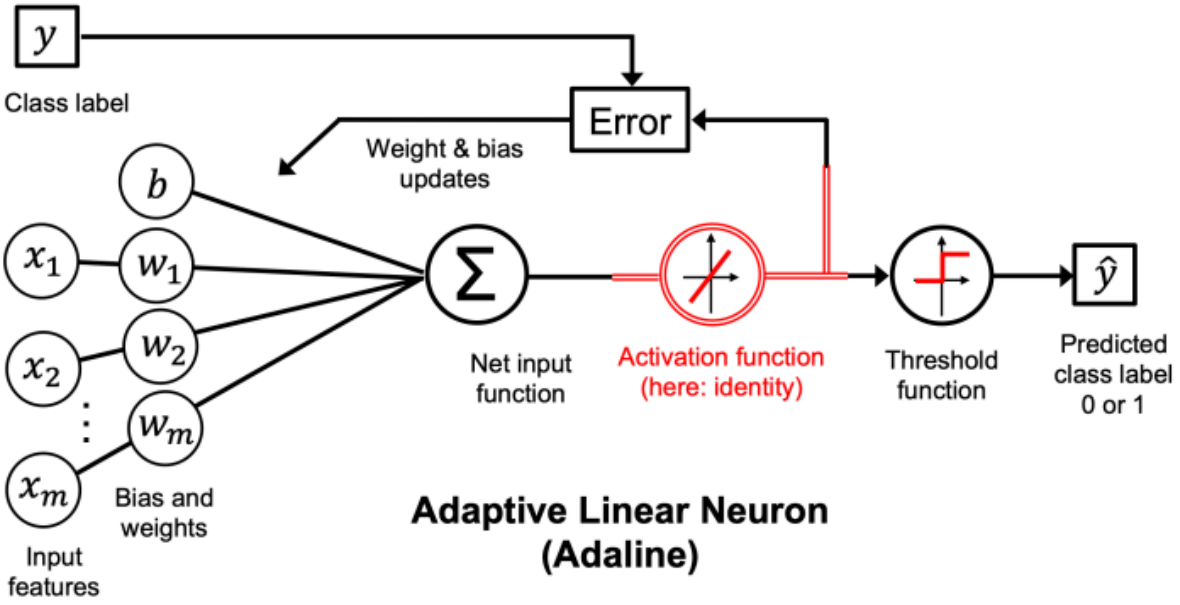
The Perceptron uses a step activation function to determine the predicted output:

$$\hat{y} = \sigma(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$



ADAptive LInear NEuron (Adaline)

ADAptive LInear NEuron (Adaline), is particularly useful for binary classification tasks and is the foundation for understanding more complex neural networks. Unlike the Perceptron, Adaline uses a continuous activation function, enabling it to optimize weights using gradient-based methods.



Artificial neurons

The net input z is computed as:

$$z = \sum_j w_j x_j + b = \mathbf{w}^T \mathbf{x} + b$$

Where:

- $\mathbf{x} = [x_1 \dots x_m]$ represents the input features.
- $\mathbf{w} = [w_1 \dots w_m]$ are the weights.
- b is the bias term.

What is net input?

- It is a linear combination of the weights that are connecting the input layer to the output layer.
- It can be written as a linear (identity) activation function: $\sigma(z) = z$ for Adaline.
- It is also the predicted output $\hat{y} = \sigma(z)$ for Adaline.

Update rule

In every training epoch, we updated the **weight vector** $\mathbf{w}$ and **bias unit** b using the following update rule:

$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, \quad b := b + \Delta b$$

Let's first define the **loss function** that is used in the update rule in Adaline:

$$L(\mathbf{w}, b) = \frac{1}{2n} \sum_i \left(y^{(i)} - \sigma(z)^{(i)} \right)^2$$

- Adaline uses **Mean Square Error (MSE)** as its loss function. MSE ensures smooth and continuous error gradients, making it suitable for gradient-based optimization.
  - $y^{(i)}$ is the target class label of a particular sample $x^{(i)}$.
  - Recall that $\sigma(z)^{(i)}$ is equivalent to the predicted output $\hat{y}^{(i)}$.
  - Note that multiplying $\frac{1}{2}$ is the common convention of MSE.

We should also know what is **learning rate**, η:

- Typically set it between 0.0 and 1.0.
- Set it with considering the trade-off between learning speed and overshooting the global minima.

Now, we can update the weight vector by leveraging the negative gradient of the loss function multiplied by the learning rate:

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L(\mathbf{w}, b) = -\eta \frac{\partial L(\mathbf{w}, b)}{\partial \mathbf{w}}$$

Let's have a closer look at the weight of each update, here only presents the negative gradient of the loss function:

$$\begin{aligned}
\frac{\partial L}{\partial w_j} &= \frac{\partial}{\partial w_j} \frac{1}{2n} \sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right) \right)^2 = \frac{1}{2n} \frac{\partial}{\partial w_j} \sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right) \right)^2 \\
&= \frac{1}{n} \sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right) \right) \frac{\partial}{\partial w_j} \left(y^{(i)} - \sigma\left(z^{(i)}\right) \right) \\
&= \frac{1}{n} \sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right) \right) \frac{\partial}{\partial w_j} \left(y^{(i)} - \sum_j \left(w_j x_j^{(i)} + b \right) \right) \\
&= \frac{1}{n} \sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right) \right) \left(-x_j^{(i)} \right) = -\frac{1}{n} \sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right) \right) x_j^{(i)}
\end{aligned}$$

Here is how we update the bias unit:

$$\Delta b = -\eta \nabla_b L(\mathbf{w}, b) = -\eta \frac{\partial L(\mathbf{w}, b)}{\partial b} = -\eta \frac{1}{n} \sum_i \left(y^{(i)} - \sigma(z)^{(i)} \right)$$

Threshold function

While we used the activation $\sigma(\cdot)$ to compute the gradient update. In Adaline, we implemented a threshold function to squash the continuous-valued output into binary class labels for prediction:

$$\hat{y} = \sigma(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Perceptron vs. Adaline

1. Activation Function
   - **Perceptron:** Uses a step activation function to produce binary outputs $\{0, 1\}$.
   - **Adaline:** Uses a linear (identity) activation function, $\sigma(z) = z$, to compute continuous outputs.
2. Learning Rule
   - **Perceptron:** Updates weights only for misclassified samples using the formula:

     $$\Delta w_j = \eta \cdot (y - \hat{y}) \cdot x_j$$

   - **Adaline:** Uses gradient descent to minimize the Mean Squared Error (MSE), updating weights based on:

     $$\Delta w_j = -\eta \frac{\partial L(\mathbf{w}, b)}{\partial w_j}$$

3. Loss Function
    - **Perceptron:** No explicit loss function. Learning stops when all samples are correctly classified.
    - **Adaline:** Minimizes a continuous loss function (MSE), enabling smooth weight updates.
4. Convergence
    - **Perceptron:** Converges only for linearly separable data.
    - **Adaline:** Can find the optimal solution for linear separability but may struggle with non-linear data due to its linear activation function.
5. Practical Use
    - **Perceptron:** Simple to implement but limited to linearly separable problems.
    - **Adaline:** More robust for continuous outputs and gradient-based optimization, paving the way for more advanced models like logistic regression and neural networks.

Optimization in machine learning

In deep learning, the process typically starts by defining a loss function, which helps us measure how far off the model's predictions are from the actual results. Once that's set, an optimization algorithm is used to adjust the model's parameters and work towards reducing the loss as much as possible.

Challenges

Some of the most common challenges are local minima, saddle points, and vanishing gradients.

Local minima

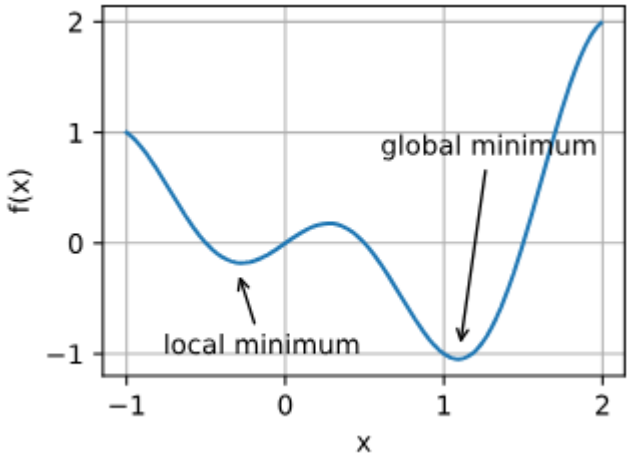
What are local minima and global minima?
For any objective function $f(x)$,

- it could be a **local minima**, if the value of $f(x)$ at x is smaller than the values of $f(x)$ at any other points in the vicinity of x;
- it is a **global minima**, if the value of $f(x)$ at x is the minimum of the objective function over the entire domain.

For example, given the function:

$$f(x) = x \cdot \cos(\pi x) \text{ for } -1.0 \leq x \leq 2.0,$$

we can approximate the local minimum and global minimum of this function.



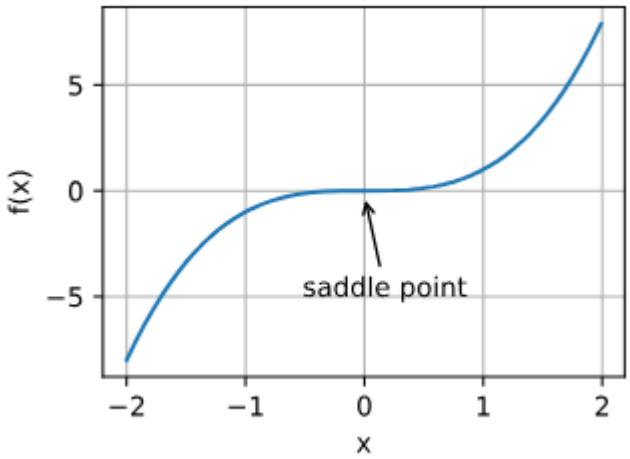
Deep learning objective functions often have a lot of local optima. When solving these optimization problems, if the solution gets close to one of these local optima, the final result from the optimization might only minimize the function in that specific region, not across the whole space. This happens because the gradient of the function tends to shrink or even hit zero near a local optima.

Another reason for gradients to vanish is saddle points.

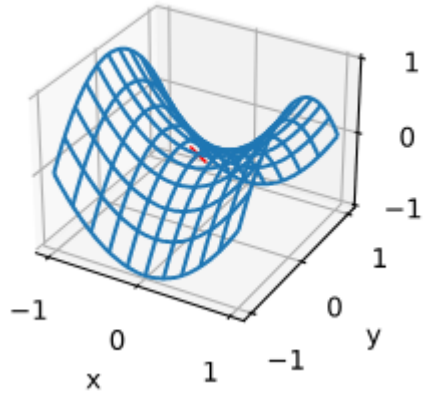
Saddle point

A **saddle point** is any location where all gradients of a function vanish but which is neither a global nor a local minimum.

For example, the function $f(x) = x^3$. Its first and second derivative vanish for $x = 0$. Optimization might stall at this point, even though it is not a minimum.



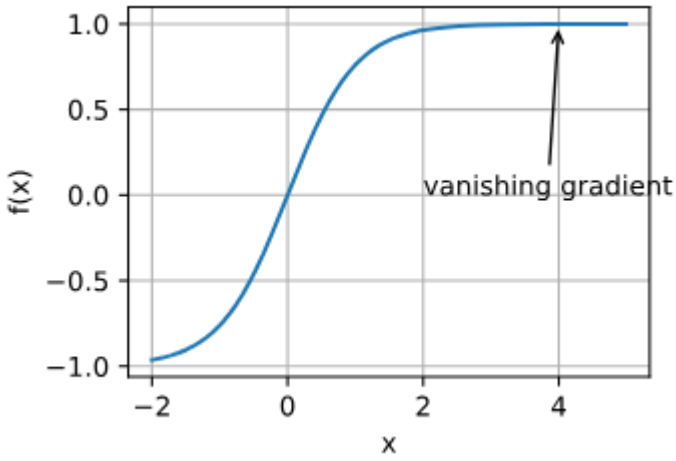
Another example in a higher dimensional space, the function $f(x, y) = x^2 - y^2$. It has its saddle point at $(0, 0)$. This is a maximum with respect to y and a minimum with respect to x.



For high-dimensional problems, typically have saddle points more likely than local minima.

Vanishing gradient

Vanishing gradient is commonly happened in the derivatives of activation functions. For instance, assume that we want to minimize the function $f(x) = \tanh(x)$ and we happen to get started at $x = 4$. As we can see, the gradient of f is close to nil. More specifically, $f'(x) = 1 - \tanh^2(x)$ and thus $f'(4) = 0.0013$. Consequently, optimization will get stuck for a long time before we make progress.



Convexity

Convexity is important when designing optimization algorithms because it makes things way easier to analyze and test. Think of it this way: if an algorithm struggles with a convex problem, which is the simpler case, it's probably not going to do well on harder problems. Even though deep learning problems are usually nonconvex, they often behave like convex problems when you're close to a local minimum.

Gradient Descent and its Variants

Gradient Descent

It is also known as **Full-batch Gradient Descent**. Full batch gradient descent is a method of finding the global minimum of a loss function by taking steps in the direction of the gradient, calculated over the entire training set.

Gradient Descent Algorithm on Adaline

Repeat until convergence:

$$w_j = w_j + \Delta w_j = w_j - \eta \frac{\partial L(\mathbf{w}, b)}{\partial w_j} = w_j - \eta \frac{1}{n} \sum_i \left(y^{(i)} - \sigma(z)^{(i)}\right) x_j^{(i)}$$

$$b_j = b_j + \Delta b_j = b_j - \eta \frac{\partial L(\mathbf{w}, b)}{\partial b} = b_j - \eta \frac{1}{n} \sum_i \left(y^{(i)} - \sigma(z)^{(i)}\right)$$

Now, let's dive into the math how gradient descent is formulized as an optimization problem and how tuning the learning rate impacts the optimization process.

One-dimensional gradient descent

Let's first look at one-dimensional gradient descent, i.e., one input feature x only.

Consider continuously differentiable real-valued objective function $f : \mathbb{R} \to \mathbb{R}$. Using a Taylor expansion we obtain the first-order approximation:

$$f(x + \epsilon) = f(x) + \epsilon f'(x) + \mathcal{O}\left(\epsilon^2\right)$$

We can assume that for small ϵ moving in the direction of the negative gradient will decrease f. To keep things simple we pick a fixed step size $\eta > 0$ (i.e., learning rate) and choose $\epsilon = -\eta f'(x)$. Plugging this into the Taylor expansion above we get:

$$f\left(x - \eta f'(x)\right) = f(x) - \eta f'^2(x) + \mathcal{O}\left(\eta^2 f'^2(x)\right)$$

If the derivative $f'(x) \neq 0$ does not vanish we make progress since $\eta f'^2(x) > 0$. Moreover, we can always choose η small enough for the higher-order terms to become irrelevant. Hence we arrive at

$$f\left(x - \eta f'(x)\right) \lesssim f(x)$$

This means that, if we use

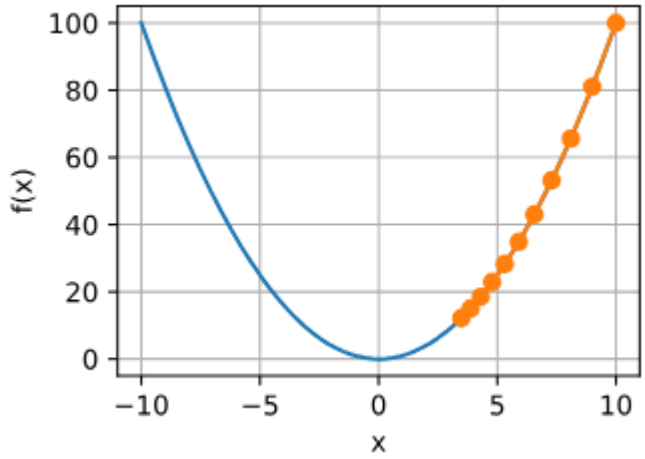
$$x \leftarrow x - \eta f'(x)$$

to iterate x, the value of function $f(x)$ might decline. Therefore, in gradient descent we first choose an initial value x and a constant $\eta > 0$ and then use them to continuously iterate x until the stop condition is reached, for example, when the magnitude of the gradient $|f'(x)|$ is small enough or the number of iterations has reached a certain value.

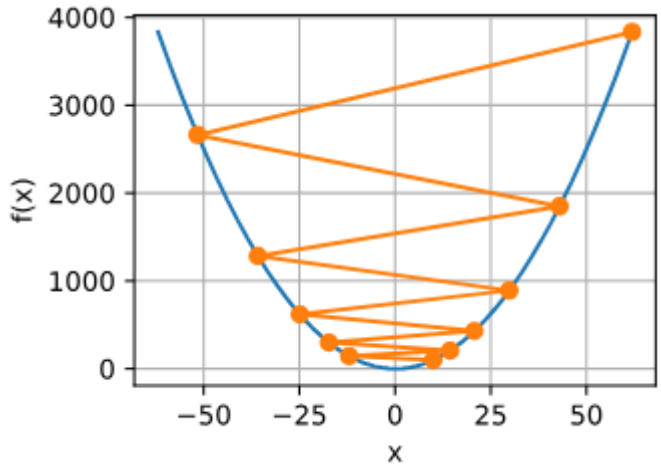
Learning rate impacts performance and efficiency

For example, we have an objective function $f(x) = x^2$, and we are going to implement gradient descent. We are also going to illustrate how the learning rate impacts the convergence speed.

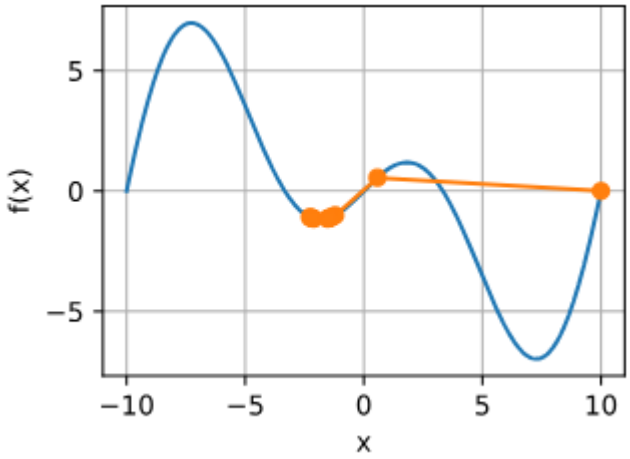
- If the learning rate is too small, the model will update its parameters really slowly. This means it'll take way more iterations to get closer to a good solution. Here is the plot when $\eta = 0.005$:



- If the learning rate is too large, $|\eta f'(x)|$ might be too large for the first-order Taylor expansion formula. That is, the term $\mathcal{O}\left(\eta^2 f'^2(x)\right)$ might become significant. In this case, we cannot guarantee that the iteration of x will be able to lower the value of $f(x)$. Here is the plot when $\eta = 1.1$, x overshoots the optimal solution $x = 0$ and gradually diverges to $x = 61$.



Another example for a nonconvex function, $f(x) = x \cdot \cos(cx)$, for some constant c. This function has infinitely many local minima, so we may have many solutions. Here, we illustrate high learning rate can lead to a poor local minima:



Multivariate gradient descent

Now, let's move forward to multivariate gradient descent, considering $\mathbf{x} = [x_1, x_2, \ldots, x_d]^\top$. That is, the objective function $f : \mathbb{R}^d \to \mathbb{R}$ maps vectors into scalars. Correspondingly its gradient is multivariate, too. It

is a vector consisting of d partial derivatives:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f(\mathbf{x})}{\partial x_1}, \frac{\partial f(\mathbf{x})}{\partial x_2}, \ldots, \frac{\partial f(\mathbf{x})}{\partial x_d}\right]^\top$$

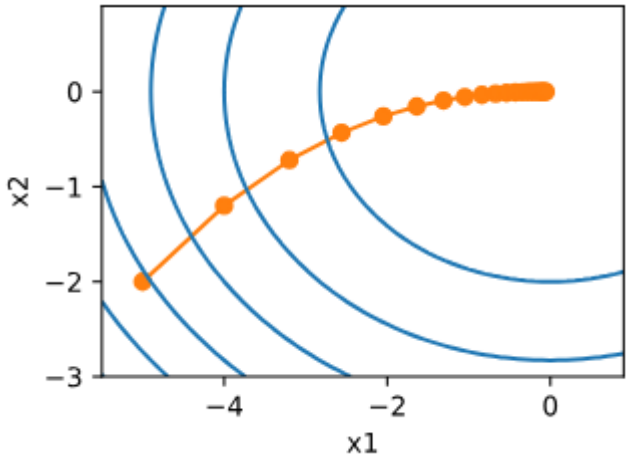
Each partial derivative element $\partial f(\mathbf{x})/\partial x_i$ in the gradient indicates the rate of change of f at $\mathbf{x}$ with respect to the input x_i. As before, we can use the corresponding Taylor approximation for multivariate functions to get some idea of what we should do. In particular, we have that

$$f(\mathbf{x} + \boldsymbol{\epsilon}) = f(\mathbf{x}) + \epsilon^\top \nabla f(\mathbf{x}) + \mathcal{O}\left(\|\boldsymbol{\epsilon}\|^2\right)$$

In other words, up to second-order terms in $\boldsymbol{\epsilon}$ the direction of steepest descent is given by the negative gradient $-\nabla f(\mathbf{x})$. Choosing a suitable learning rate $\eta > 0$ yields the prototypical gradient descent algorithm:

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f(\mathbf{x})$$

For example, an objective function $f(\mathbf{x}) = x_1^2 + 2x_2^2$ with a two-dimensional vector $\mathbf{x} = [x_1, x_2]^\top$ as input and a scalar as output. The gradient is given by $\nabla f(\mathbf{x}) = [2x_1, 4x_2]^\top$. We will observe the trajectory of $\mathbf{x}$ by gradient descent from the initial position $[-5, -2]$. Here is the illustration:



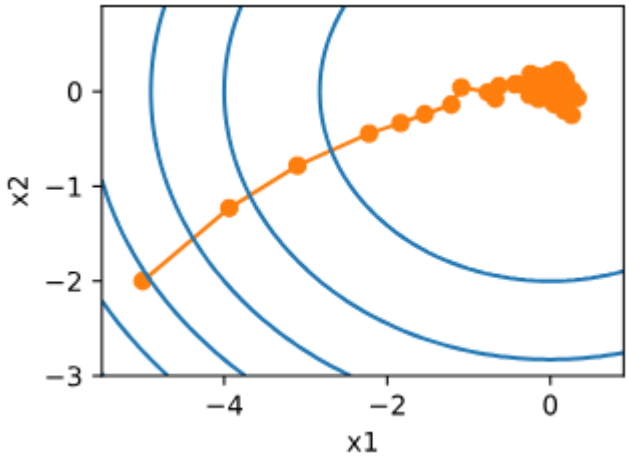
Stochastic Gradient Descent (SGD)

Stochastic gradient descent (SGD) is a variant of gradient descent where the weights are updated incrementally for each training example, rather than based on the sum of the accumulated error over all training examples. Usually comparing to Gradient Descent, it is faster but introduces noise in the updates.

Stochastic Gradient Descent Algorithm on Adaline

Repeat until convergence:

$$w_j = w_j + \Delta w_j = w_j - \eta \left(y^{(i)} - \sigma(z)^{(i)}\right) x_j^{(i)}$$
$$b_j = b_j + \Delta b_j = b_j - \eta \left(y^{(i)} - \sigma(z)^{(i)}\right)$$



Mini-batch Gradient Descent

Gradient descent isn’t great at handling similar data efficiently—it ends up doing a lot of redundant work. On the other hand, stochastic gradient descent isn’t very computationally efficient because CPUs and GPUs can’t fully take advantage of vectorization to speed things up. Thus, there is Mini-batch Gradient Descent, a compromise between these two.

It applies full-batch gradient descent to smaller subsets of the training data, such as 32 training examples at a time. It converges faster than full batch gradient descent and allows for more computational efficiency by using vectorized operations to replace the "for loop" over the training examples used in SGD.

Mathematically, when updating the weights, we can formulate as:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta_t \mathbf{g}_t$$

Where the gradient $\mathbf{g}_t$ can be defined as:

$$\mathbf{g}_t = \partial_{\mathbf{w}} \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} f\left(\mathbf{x}_i, \mathbf{w}\right)$$

After understanding the math behind basic neural networks and gradient descent, let's check out the code implementation!